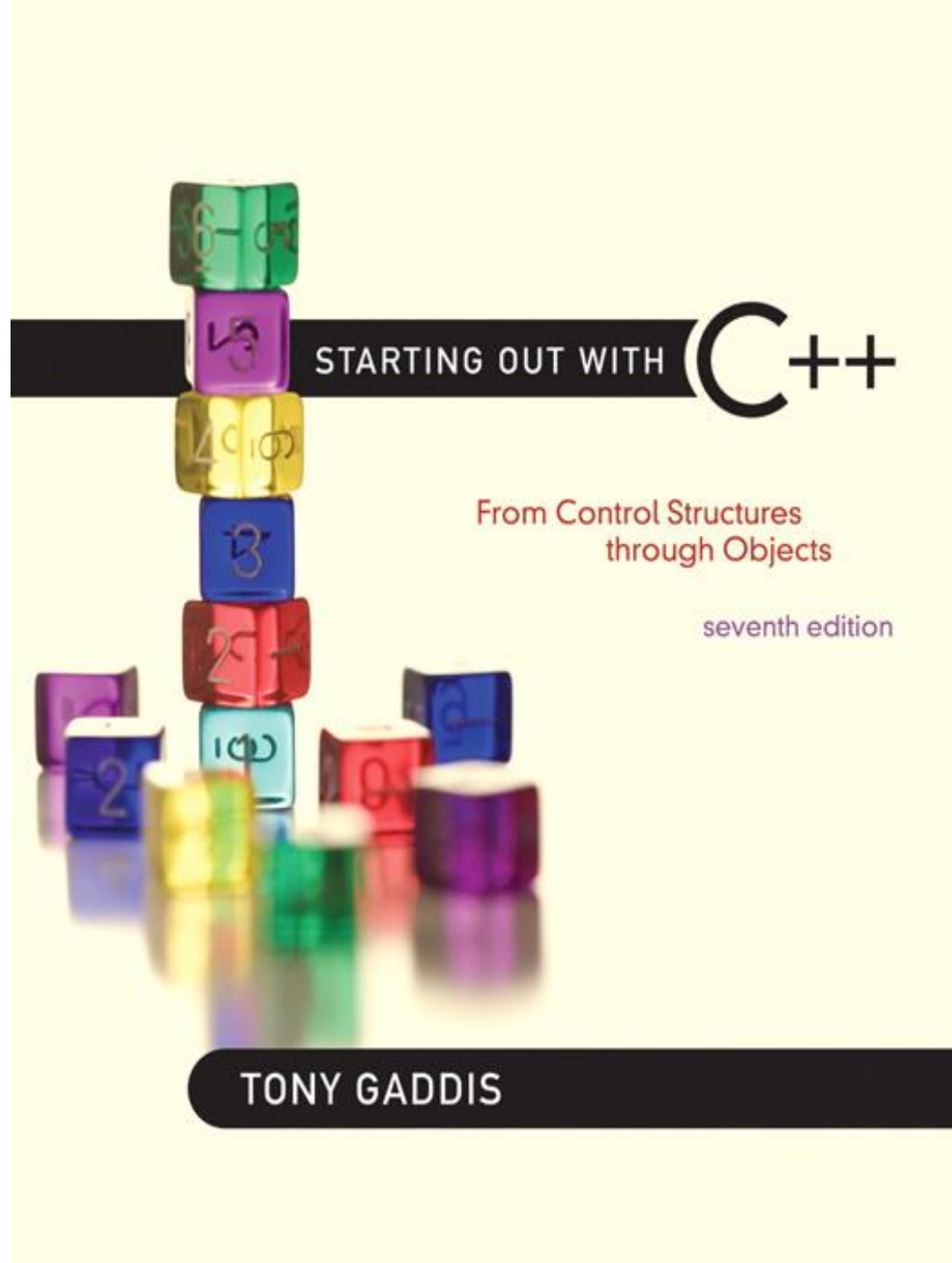


Chapter 9:

Pointers

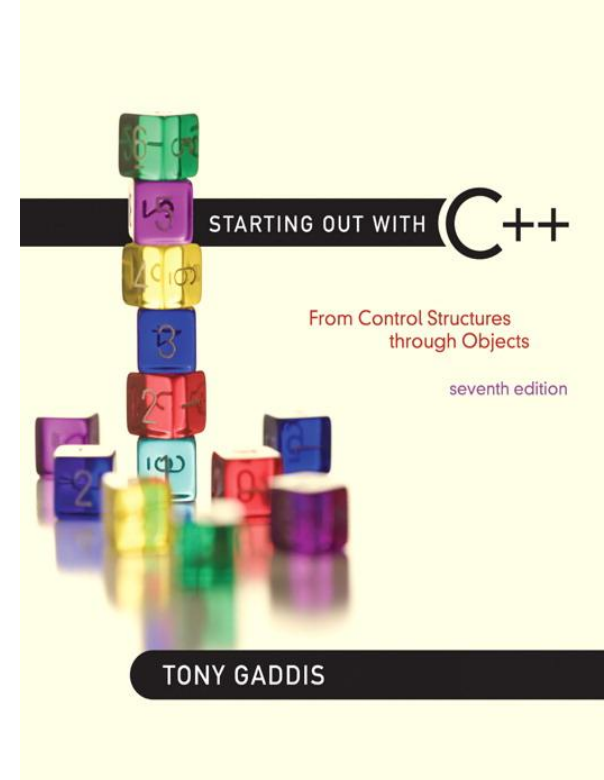


Addison-Wesley
is an imprint of

PEARSON

Copyright © 2012 Pearson Education, Inc.

9.1



Getting the Address of a Variable

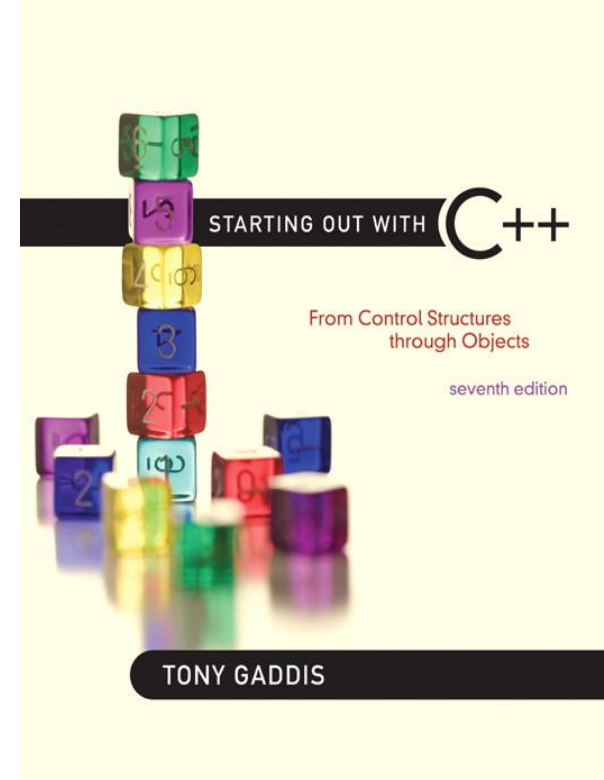
Getting the Address of a Variable

- Each variable in program is stored at a unique address
- Use address operator & to get address of a variable:

```
int num = -99;  
cout << &num; // prints address  
               // in hexadecimal
```

9.2

Pointer Variables



Pointer Variables

- Pointer variable : Often just called a pointer, it's a variable that holds an address
- Because a pointer variable holds the address of another piece of data, it "points" to the data

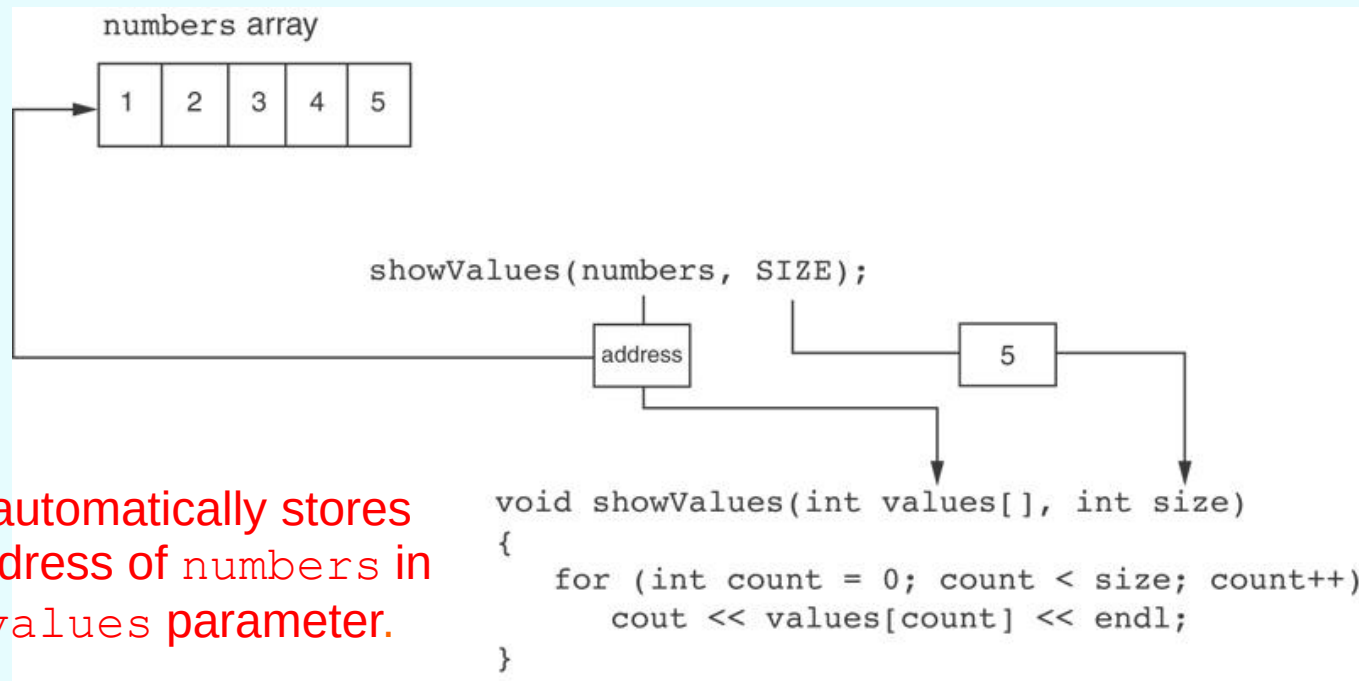
Something Like Pointers: Arrays

- We have already worked with something similar to pointers, when we learned to pass arrays as arguments to functions.
- For example, suppose we use this statement to pass the array `numbers` to the `showValues` function:

```
showValues (numbers, SIZE) ;
```

Something Like Pointers : Arrays

The `values` parameter, in the `showValues` function, points to the `numbers` array.



C++ automatically stores the address of `numbers` in the `values` parameter.

Something Like Pointers: Reference Variables

- We have also worked with something like pointers when we learned to use reference variables. Suppose we have this function:

```
void getOrder(int &donuts)
{
    cout << "How many doughnuts do you want? ";
    cin >> donuts;
}
```

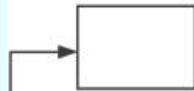
- And we call it with this code:

```
int jellyDonuts;
getOrder(jellyDonuts);
```

Something Like Pointers: Reference Variables

The `donuts` parameter, in the `getOrder` function, points to the `jellyDonuts` variable.

`jellyDonuts` variable



`getOrder(jellyDonuts);`

address

`void getOrder(int &donuts)`

```
{  
    cout << "How many doughnuts do you want? ";  
    cin >> donuts;  
}
```

C++ automatically stores
the address of
`jellyDonuts` in the
`donuts` parameter.

Pointer Variables

- Pointer variables are yet another way using a memory address to work with a piece of data.
- Pointers are more "low-level" than arrays and reference variables.
- This means you are responsible for finding the address you want to store in the pointer and correctly using it.

Pointer Variables

- Definition:

```
int *intptr;
```

- Read as:

“intptr can hold the address of an int”

- Spacing in definition does not matter:

```
int * intptr;    // same as above
```

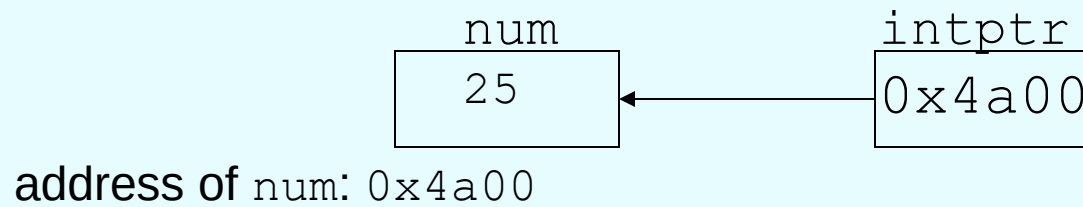
```
int*  intptr;    // same as above
```

Pointer Variables

- Assigning an address to a pointer variable:

```
int *intptr;  
intptr = &num;
```

- Memory layout:



Program 9-2

```
1  // This program stores the address of a variable in a pointer.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      int x = 25;      // int variable
8      int *ptr;        // Pointer variable, can point to an int
9
10     ptr = &x;        // Store the address of x in ptr
11     cout << "The value in x is " << x << endl;
12     cout << "The address of x is " << ptr << endl;
13     return 0;
14 }
```

Program Output

The value in x is 25

The address of x is 0x7e00

The Indirection Operator

- The indirection operator (*) dereferences a pointer.
- It allows you to access the item that the pointer points to.

```
int x = 25;  
int *intptr = &x;  
cout << *intptr << endl;
```



This prints 25.

Program 9-3

```
1 // This program demonstrates the use of the indirection operator.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int x = 25;    // int variable
8     int *ptr;      // Pointer variable, can point to an int
9
10    ptr = &x;      // Store the address of x in ptr
11
12    // Use both x and ptr to display the value in x.
13    cout << "Here is the value in x, printed twice:\n";
14    cout << x << endl;    // Displays the contents of x
15    cout << *ptr << endl; // Displays the contents of x
16
17    // Assign 100 to the location pointed to by ptr. This
18    // will actually assign 100 to x.
19    *ptr = 100;
20
21    // Use both x and ptr to display the value in x.
22    cout << "Once again, here is the value in x:\n";
23    cout << x << endl;    // Displays the contents of x
24    cout << *ptr << endl; // Displays the contents of x
25    return 0;
26 }
```

Program 9-3

(continued)

Program Output

```
Here is the value in x, printed twice:
```

```
25
```

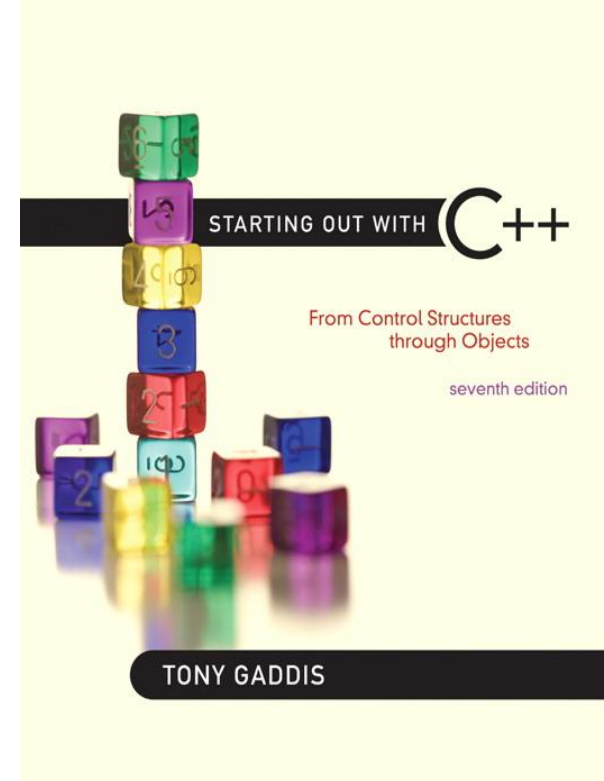
```
25
```

```
Once again, here is the value in x:
```

```
100
```

```
100
```

9.3



The Relationship Between Arrays and Pointers

The Relationship Between Arrays and Pointers

- Array name is starting address of array

```
int vals[] = {4, 7, 11};
```

4	7	11
---	---	----

starting address of `vals`: 0x4a00

```
cout << vals;           // displays  
                        // 0x4a00  
cout << vals[0];       // displays 4
```

The Relationship Between Arrays and Pointers

- Array name can be used as a pointer constant:

```
int vals[] = {4, 7, 11};  
cout << *vals;    // displays 4
```

- Pointer can be used as an array name:

```
int *valptr = vals;  
cout << valptr[1]; // displays 7
```

Program 9-5

```
1  // This program shows an array name being dereferenced with the *
2  // operator.
3  #include <iostream>
4  using namespace std;
5
6  int main()
7  {
8      short numbers[] = {10, 20, 30, 40, 50};
9
10     cout << "The first element of the array is ";
11     cout << *numbers << endl;
12     return 0;
13 }
```

Program Output

The first element of the array is 10

Lab: pointer and array

- Define a pointer p points to array numbers in Program 9.5
- Run a loop to display all array elements using pointer p and subscripts
- `short * p = number; // p points to number`
- `for(int i=0; i < 5; i++)`
 `cout << p[i] << " ";`
 `cout << endl;`

Pointers in Expressions

Given:

```
int vals[]={4,7,11}, *valptr;  
valptr = vals;
```

What is `valptr + 1`? It means (address in
`valptr`) + (1 * size of an int)

```
cout << *(valptr+1); //displays 7  
cout << *(valptr+2); //displays 11
```

Must use () as shown in the expressions

Array Access

- Array elements can be accessed in many ways:

Array access method	Example
array name and []	<code>vals[2] = 17;</code>
pointer to array and []	<code>valptr[2] = 17;</code>
array name and subscript arithmetic	<code>*(vals + 2) = 17;</code>
pointer to array and subscript arithmetic	<code>*(valptr + 2) = 17;</code>

Array Access

- Conversion: `vals[i]` is equivalent to `*(vals + i)`
- No bounds checking performed on array access, whether using array name or a pointer

From Program 9-7

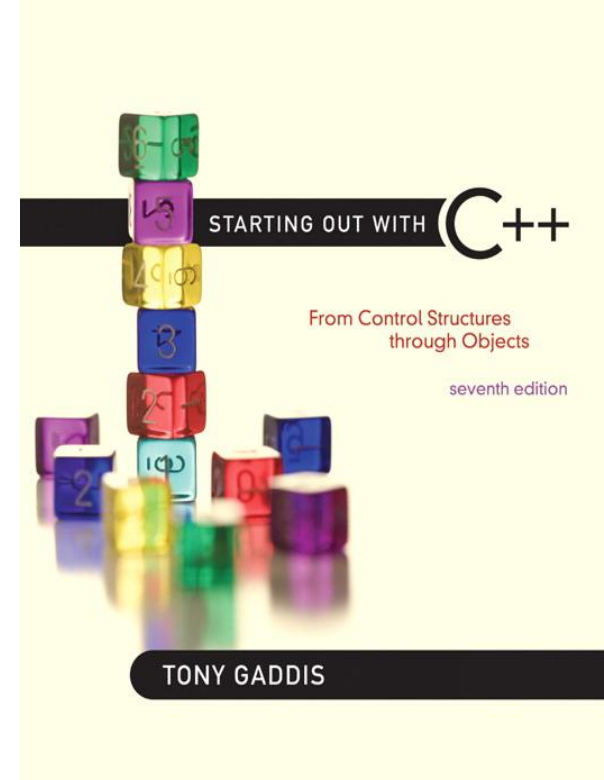
```
9      const int NUM_COINS = 5;
10     double coins[NUM_COINS] = {0.05, 0.1, 0.25, 0.5, 1.0};
11     double *doublePtr;    // Pointer to a double
12     int count;            // Array index
13
14     // Assign the address of the coins array to doublePtr.
15     doublePtr = coins;
16
17     // Display the contents of the coins array. Use subscripts
18     // with the pointer!
19     cout << "Here are the values in the coins array:\n";
20     for (count = 0; count < NUM_COINS; count++)
21         cout << doublePtr[count] << " ";
22
23     // Display the contents of the array again, but this time
24     // use pointer notation with the array name!
25     cout << "\nAnd here they are again:\n";
26     for (count = 0; count < NUM_COINS; count++)
27         cout << *(coins + count) << " ";
28     cout << endl;
```

Program Output

```
Here are the values in the coins array:
0.05 0.1 0.25 0.5 1
And here they are again:
0.05 0.1 0.25 0.5 1
```

9.4

Pointer Arithmetic



Pointer Arithmetic

- Operations on pointer variables:

Operation	Example
	<pre>int vals[]={4,7,11}; int *valptr = vals;</pre>
<code>++, --</code>	<pre>valptr++; // points at 7 valptr--; // now points at 4</pre>
<code>+, - (pointer and int)</code>	<pre>cout << *(valptr + 2); // 11</pre>
<code>+=, -= (pointer and int)</code>	<pre>valptr = vals; // points at 4 valptr += 2; // points at 11</pre>
<code>- (pointer from pointer)</code>	<pre>cout << valptr-vals; // difference // (number of ints) between valptr // and val</pre>

From Program 9-9

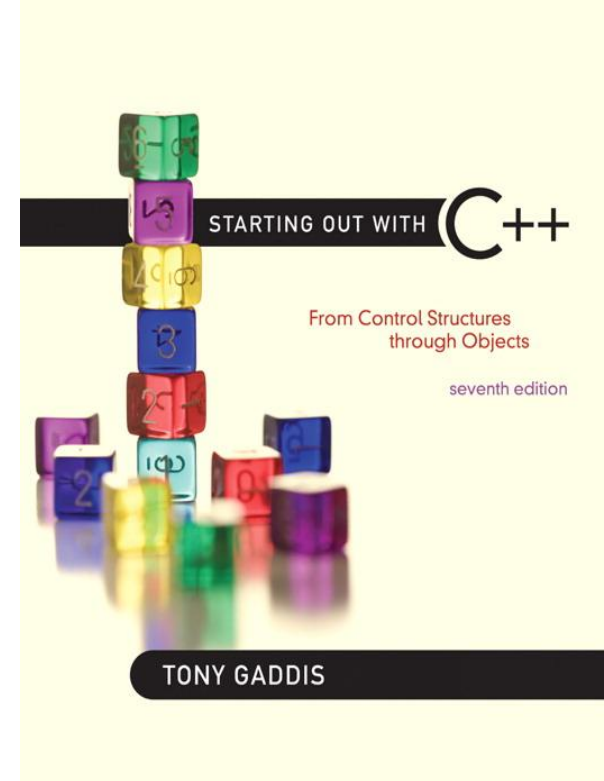
```
7     const int SIZE = 8;
8     int set[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
9     int *numPtr;    // Pointer
10    int count;      // Counter variable for loops
11
12    // Make numPtr point to the set array.
13    numPtr = set;
14
15    // Use the pointer to display the array contents.
16    cout << "The numbers in set are:\n";
17    for (count = 0; count < SIZE; count++)
18    {
19        cout << *numPtr << " ";
20        numPtr++;
21    }
22
23    // Display the array contents in reverse order.
24    cout << "\nThe numbers in set backward are:\n";
25    for (count = 0; count < SIZE; count++)
26    {
27        numPtr--;
28        cout << *numPtr << " ";
29    }
```

Program Output

```
The numbers in set are:
5 10 15 20 25 30 35 40
The numbers in set backward are:
40 35 30 25 20 15 10 5
```

9.5

Initializing Pointers



Initializing Pointers

- Can initialize at definition time:

```
int num, *numptr = &num;  
int val[3], *valptr = val;
```

- Cannot mix data types:

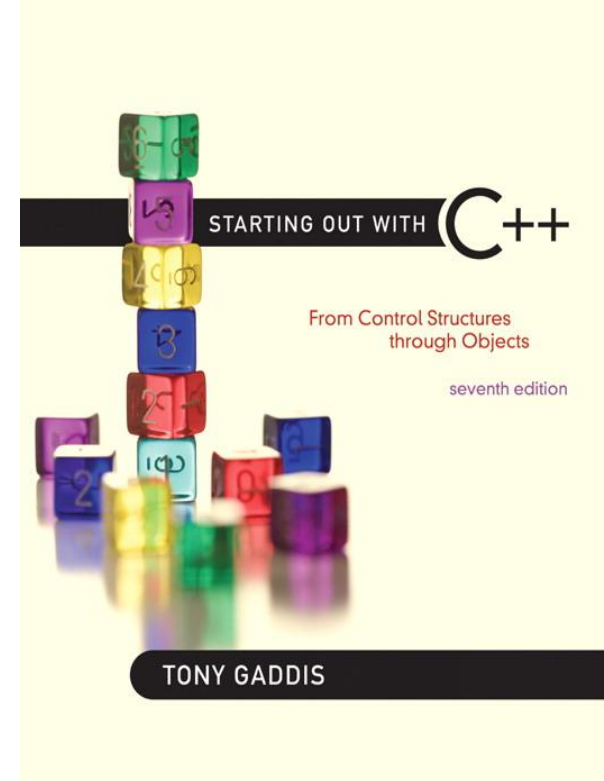
```
double cost;  
int *ptr = &cost; // won't work
```

- Can test for an invalid address for `ptr` with:

```
if (!ptr) ...
```

9.6

Comparing Pointers

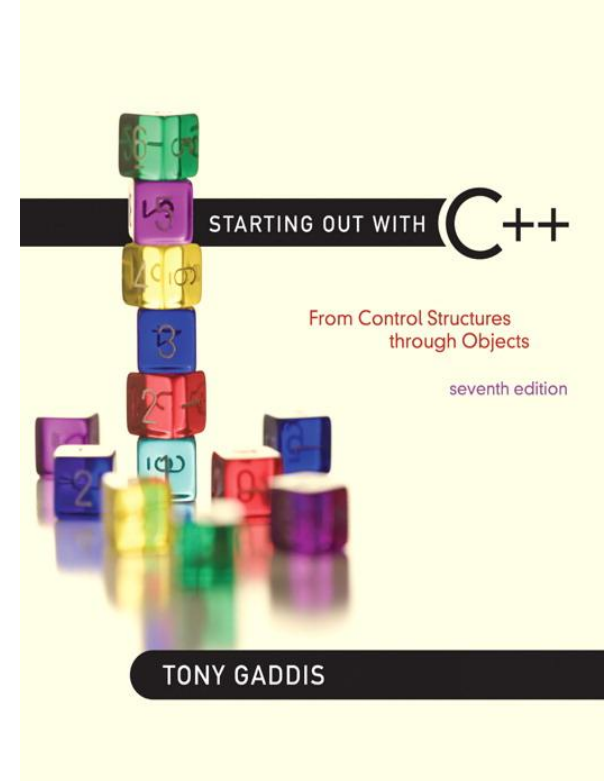


Comparing Pointers

- Relational operators (<, >=, etc.) can be used to compare addresses in pointers
- Comparing addresses in pointers is not the same as comparing contents pointed at by pointers:

```
if (ptr1 == ptr2)    // compares
                     // addresses
if (*ptr1 == *ptr2) // compares
                     // contents
```

9.7



Pointers as Function Parameters

Pointers as Function Parameters

- A pointer can be a parameter
- Works like reference variable to allow change to argument from within function
- Requires:
 - 1) asterisk * on parameter in prototype and heading
`void getNum(int *ptr); // ptr is pointer to an int`
 - 2) asterisk * in body to dereference the pointer
`cin >> *ptr;`
 - 3) address as argument to the function
`getNum(&num); // pass address of num to getNum`

Example

```
void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int num1 = 2, num2 = -3;
swap(&num1, &num2);
```

Program 9-11

```
1  // This program uses two functions that accept addresses of
2  // variables as arguments.
3  #include <iostream>
4  using namespace std;
5
6  // Function prototypes
7  void getNumber(int *);
8  void doubleValue(int *);
9
10 int main()
11 {
12     int number;
13
14     // Call getNumber and pass the address of number.
15     getNumber(&number);
16
17     // Call doubleValue and pass the address of number.
18     doubleValue(&number);
19
20     // Display the value in number.
21     cout << "That value doubled is " << number << endl;
22     return 0;
23 }
24
```

(Program Continues)

Program 9-11 *(continued)*

```
25  //*****
26  // Definition of getNumber. The parameter, input, is a pointer. *
27  // This function asks the user for a number. The value entered  *
28  // is stored in the variable pointed to by input.                *
29  //*****
30
31  void getNumber(int *input)
32  {
33      cout << "Enter an integer number: ";
34      cin >> *input;
35  }
36
37  //*****
38  // Definition of doubleValue. The parameter, val, is a pointer. *
39  // This function multiplies the variable pointed to by val by    *
40  // two.                                                            *
41  //*****
42
43  void doubleValue(int *val)
44  {
45      *val *= 2;
46  }
```

Program Output with Example Input Shown in Bold

Enter an integer number: **10** [Enter]
That value doubled is 20

Lab

- Write a function that takes three variable (a, b, c) in as separate parameters and rotates the values stored so that value a goes to b, b, to c and c to a. Test this function in a program

Solution

```
void swap3(int *p, int *q, int *r){
    int tmp;
    tmp= *p; *p=*q; *q=*r; *r=tmp;
}
main(){
    int a, b, c;
    printf("Enter a, b, c:");
    cin >> a >> b >> c;
    cout << "Value before swap. a="
    << a << ",b=" << b << ",c=" << c
    << endl;
    swap3(&a, &b, &c);
    cout << "Value after swap. a="
    << a << ",b=" << b << ",c=" << c
    << endl;
}
```

Lab: Function with pointers as arguments

- Extend the program 9-12 with a new function that displays outstanding sale figures (which are greater than the average sale for each quarter.)

OutstandingSales

```
void outstandingSales(double *arr, int size) {
    int count;
    double avg = totalSales(arr,size)/size;
    cout << "Outstanding quarter sales figures are:\n";
    for (count =0; count < size; count++) {
        if (arr [count] > avg {
            cout << "Quarter " << (count +1) << ":" << arr[count] <<
endl;
        }
    }
}

int main(){
    ...
    outstandingSales(sales, QTRS);
    ....
}
```

Lab 2: Bubble sort function with pointer argument

- Write a new function for implementing Bubble sort using pointer as 1st argument (instead of array).

Version with array argument

```
void bubbleSort(int array[], int size) {  
    bool swap;  
    int temp;  
    do  
    {  
        swap = false;  
        for (int count = 0; count < (size - 1); count++)  
        {  
            if (array[count] > array[count + 1])  
            {  
                temp = array[count];  
                array[count] = array[count + 1];  
                array[count + 1] = temp;  
                swap = true;  
            }  
        }  
    } while (swap);  
}
```

Pointers to Constants

- If we want to store the address of a constant in a pointer, then we need to store it in a pointer-to-const.

Pointers to Constants

- Example: Suppose we have the following definitions:

```
const int SIZE = 6;  
const double payRates[SIZE] =  
    { 18.55, 17.45, 12.85,  
      14.97, 10.35, 18.89 };
```

- In this code, `payRates` is an array of constant doubles.

Pointers to Constants

- Suppose we wish to pass the `payRates` array to a function? Here's an example of how we can do it.

```
void displayPayRates(const double *rates, int size)
{
    for (int count = 0; count < size; count++)
    {
        cout << "Pay rate for employee " << (count + 1)
              << " is $" << *(rates + count) << endl;
    }
}
```

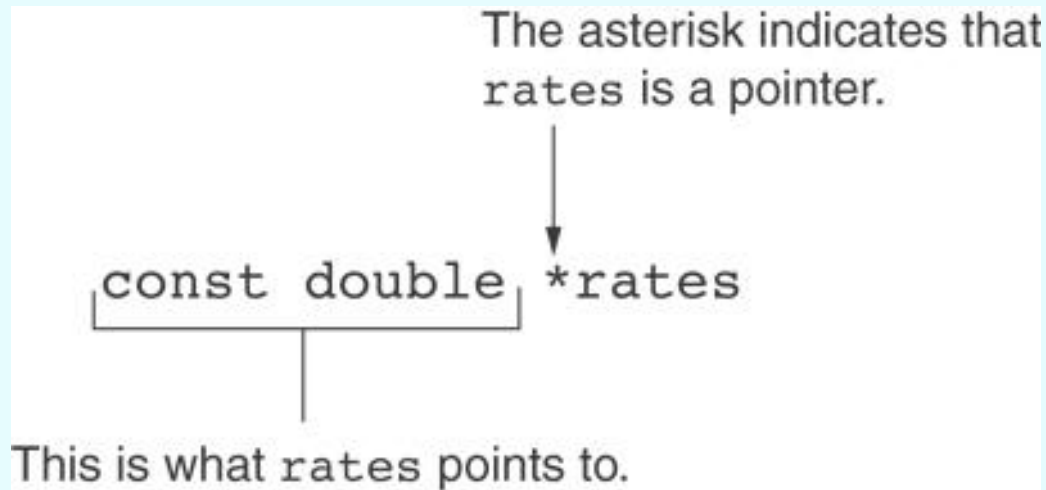
The parameter, `rates`, is a pointer to `const double`.

Declaration of a Pointer to Constant

The asterisk indicates that rates is a pointer.

`const double *rates`

This is what rates points to.

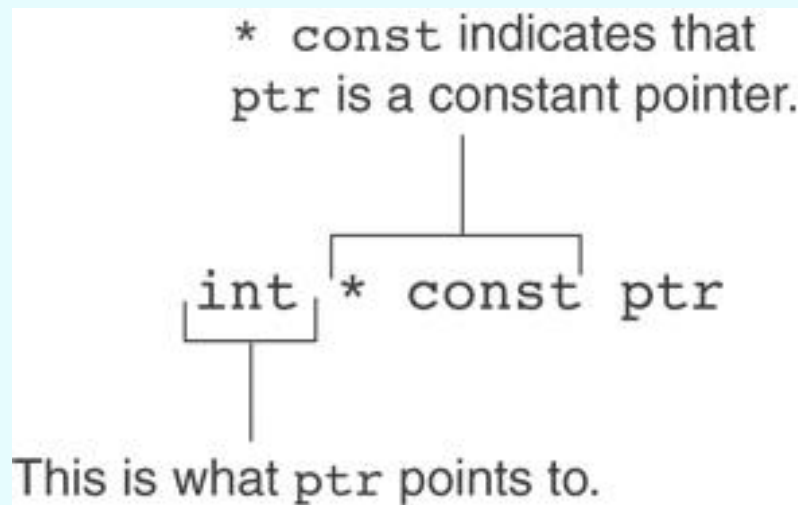


Constant Pointers

- A constant pointer is a pointer that is initialized with an address, and cannot point to anything else.
- Example

```
int value = 22;  
int * const ptr = &value;
```

Constant Pointers



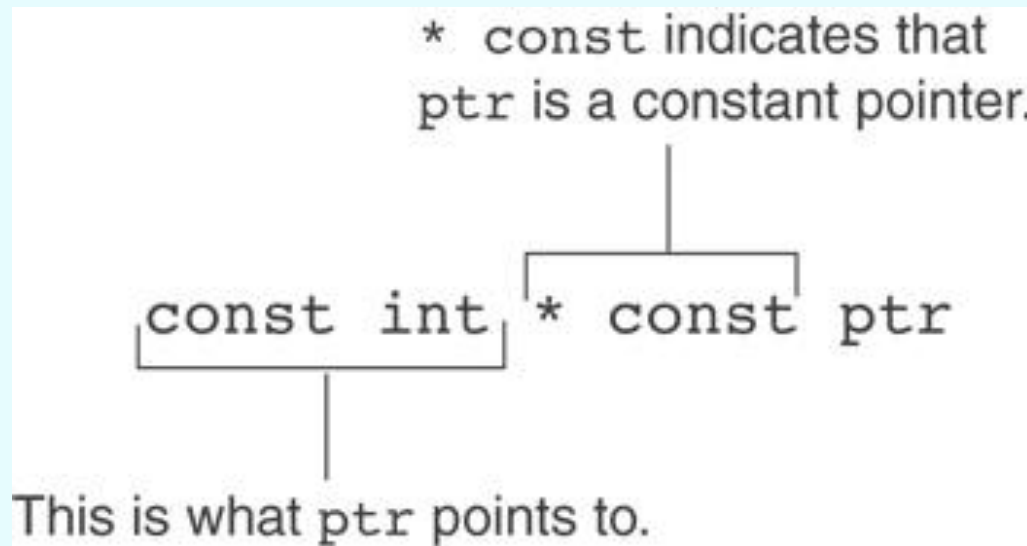
Constant Pointers to Constants

- A constant pointer to a constant is:
 - a pointer that points to a constant
 - a pointer that cannot point to anything except what it is pointing to

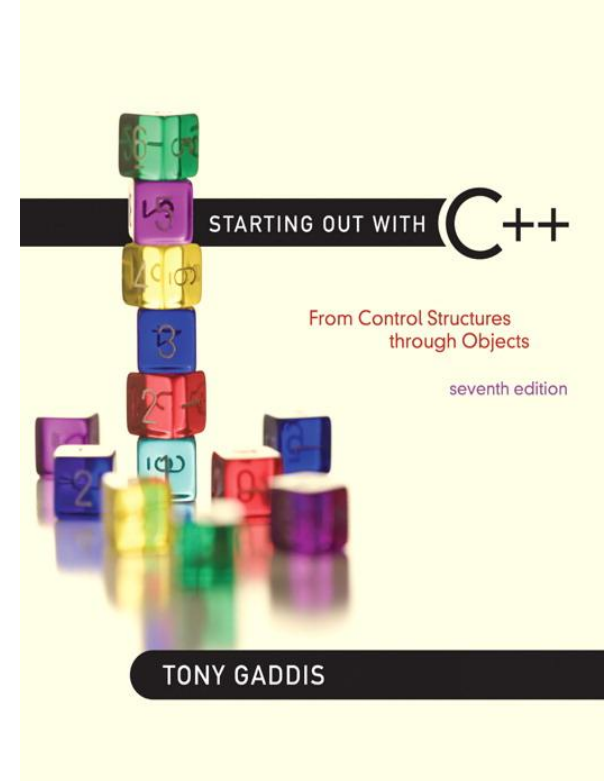
- Example:

```
int value = 22;  
const int * const ptr = &value;
```

Constant Pointers to Constants



9.8



Dynamic Memory Allocation

Dynamic Memory Allocation

- Can allocate storage for a variable while program is running
- Computer returns address of newly allocated variable
- Uses `new` operator to allocate memory:

```
double *dptr;  
dptr = new double;
```
- `new` returns address of memory location

Dynamic Memory Allocation

- Can also use `new` to allocate array:

```
const int SIZE = 25;  
arrayPtr = new double[SIZE];
```

- Can then use `[]` or pointer arithmetic to access array:

```
for(i = 0; i < SIZE; i++)  
    arrayptr[i] = i * i;
```

or

```
for(i = 0; i < SIZE; i++)  
    *(arrayptr + i) = i * i;
```

- Program will terminate if not enough memory available to allocate

Releasing Dynamic Memory

- Use `delete` to free dynamic memory:
`delete fptr;`
- Use `[]` to free dynamic array:
`delete [] arrayptr;`
- Only use `delete` with dynamic memory!

Program 9-14

```
1  // This program totals and averages the sales figures for any
2  // number of days. The figures are stored in a dynamically
3  // allocated array.
4  #include <iostream>
5  #include <iomanip>
6  using namespace std;
7
8  int main()
9  {
10     double *sales,      // To dynamically allocate an array
11           total = 0.0,  // Accumulator
12           average;      // To hold average sales
```

Program 9-14 *(continued)*

```
13      int numDays,          // To hold the number of days of sales
14          count;           // Counter variable
15
16      // Get the number of days of sales.
17      cout << "How many days of sales figures do you wish ";
18      cout << "to process? ";
19      cin >> numDays;
20
21      // Dynamically allocate an array large enough to hold
22      // that many days of sales amounts.
23      sales = new double[numDays];
24
25      // Get the sales figures for each day.
26      cout << "Enter the sales figures below.\n";
27      for (count = 0; count < numDays; count++)
28      {
29          cout << "Day " << (count + 1) << ": ";
30          cin >> sales[count];
31      }
32
```

Program 9-14 (Continued)

```
33     // Calculate the total sales
34     for (count = 0; count < numDays; count++)
35     {
36         total += sales[count];
37     }
38
39     // Calculate the average sales per day
40     average = total / numDays;
41
42     // Display the results
43     cout << fixed << showpoint << setprecision(2);
44     cout << "\n\nTotal Sales: $" << total << endl;
45     cout << "Average Sales: $" << average << endl;
46
47     // Free dynamically allocated memory
48     delete [] sales;
49     sales = 0;          // Make sales point to null.
50
51     return 0;
52 }
```

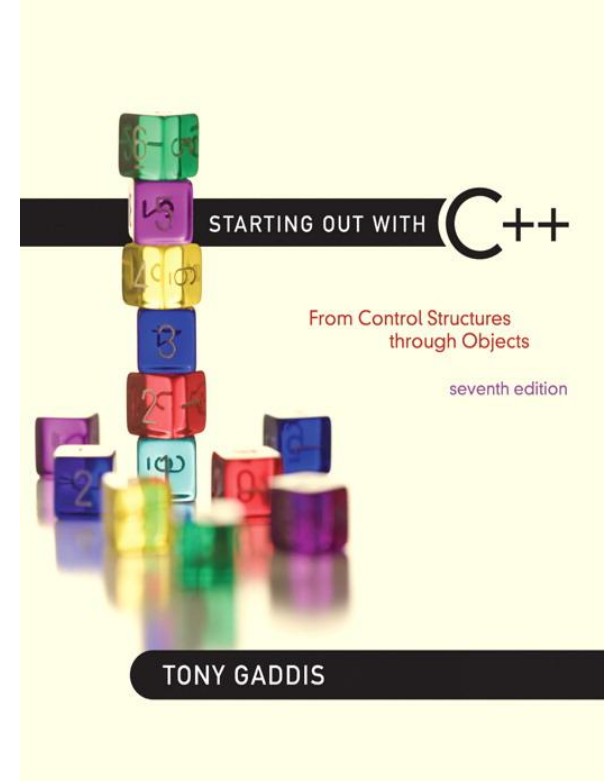
Program Output with Example Input Shown in Bold

```
How many days of sales figures do you wish to process? 5 [Enter]
Enter the sales figures below.
Day 1: 898.63 [Enter]
Day 2: 652.32 [Enter]
Day 3: 741.85 [Enter]
Day 4: 852.96 [Enter]
Day 5: 921.37 [Enter]

Total Sales: $4067.13
Average Sales: $813.43
```

Notice that in line 49 the value 0 is assigned to the `sales` pointer. It is a good practice to store 0 in a pointer variable after using `delete` on it. First, it prevents code from inadvertently using the pointer to access the area of memory that was freed. Second, it prevents errors from occurring if `delete` is accidentally called on the pointer again. The `delete` operator is designed to have no effect when used on a null pointer.

9.9



Returning Pointers from Functions

Returning Pointers from Functions

- Pointer can be the return type of a function:

```
int* newNum();
```

- The function must not return a pointer to a local variable in the function.
- A function should only return a pointer:
 - to data that was passed to the function as an argument, or
 - to dynamically allocated memory

From Program 9-15

```
34 int *getRandomNumbers(int num)
35 {
36     int *array;    // Array to hold the numbers
37
38     // Return null if num is zero or negative.
39     if (num <= 0)
40         return NULL;
41
42     // Dynamically allocate the array.
43     array = new int[num];
44
45     // Seed the random number generator by passing
46     // the return value of time(0) to srand.
47     srand( time(0) );
48
49     // Populate the array with random numbers.
50     for (int count = 0; count < num; count++)
51         array[count] = rand();
52
53     // Return a pointer to the array.
54     return array;
55 }
```

In the Spotlight: function for duplicating arrays

- Suppose you are developing a program that works with arrays of integers, and you find that you frequently need to duplicate the arrays. *Rather than rewriting the array-duplicating code each time you need it, you decide to write a function that accepts an array and its size as arguments, creates a new array that is a copy of the argument array, and returns a pointer to the new array.*

From Program 9-16

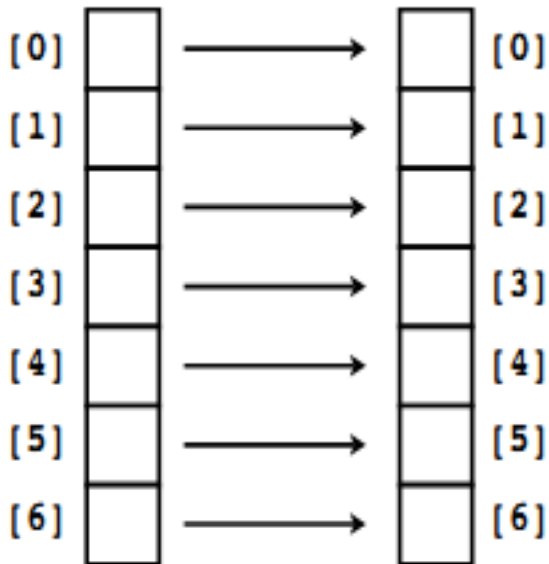
```
59  int *duplicateArray(const int *arr, int size)
60  {
61      int *newArray;
62
63      // Validate the size. If 0 or a negative
64      // number was passed, return null.
65      if (size <= 0)
66          return NULL;
67
68      // Allocate a new array.
69      newArray = new int[size];
70
71      // Copy the array's contents to the
72      // new array.
73      for (int index = 0; index < size; index++)
74          newArray[index] = arr[index];
75
76      // Return a pointer to the new array.
77      return newArray;
78 }
```

Focus on Problem Solving and Program Design

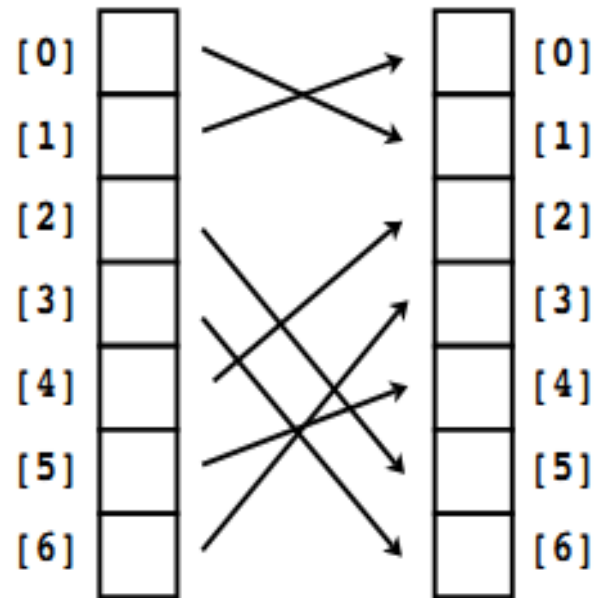
- The United Cause, a charitable relief agency, solicits donations from businesses. The local United Cause office received the following donations from the employees of CK Graphics, Inc.:
\$5, \$100, \$5, \$25, \$10, \$5, \$25, \$5, \$5, \$100, \$10, \$15,
\$10, \$5, \$10
- The donations were received in the order they appear. The United Cause manager has asked you to write a program that displays the donations in ascending order, as well as in their original order.

Strategy: Sort an array of pointers that points to donation array

arrPtr Array donations Array



arrPtr Array donations Array



Exercise on class

- Use the function getRandomNumbers above to create a dynamic array of 200000 integers. Then sort the array and display the execution time.
- Hint: `#include<ctime>`
`int start_s=clock() ;`
`//the code you wish to time goes here`
`int stop_s=clock();`
`cout<<(stop_s-`
`start_s)/double(CLOCKS_PER_SEC)*1000;`

Lab on dynamic memory allocation

- Modify Program 9-17 (the United Cause case study program) so it can be used with any set of donations. The program should dynamically allocate the donations array and ask the user to input its values.

Lab on dynamic memory allocation (2)

- Modify Program 9-17 (the United Cause case study program) so the arrptr array is sorted in descending order instead of ascending order.

Exercises of lecture

- Checkpoint 9.7, 9.8 p507
- 2.1 Write a function `arrCmp` using two pointers as arguments to compare two arrays of the same size. It returns `true` if all the elements at the same subscript in two array are identical and `false` if not.
- Demo it in your program, for example
 - `arrCmp(a,b, SIZE) → true`

Cont

- 2.2
- Write a program that use a pointer for finding the maximum and minimum element of an array. The it exchange them and display the result to screen.

Exercise

- Programming challenges 537
- 1, 2, 5 (getString), 8 Mode Function, 9 Median Function, 14